

Enunciado de Proyecto Completo: Sistema de Matricula Interactivo

Guia Practica de Desarrollo Orientado a Historias de Usuario

June 15, 2026

1 Introduccion al Proyecto

El objetivo de este proyecto es construir un **Sistema de Matricula** en consola aplicando los conceptos de modularizacion, estructuras de datos (listas y diccionarios), manejo de excepciones, bucles de control y logica de credenciales.

Para simular un entorno de desarrollo profesional, el proyecto se abordara utilizando **Historias de Usuario (User Stories)**. Esto permite descomponer una aplicacion compleja en pequenas tareas con un valor claro para el usuario final, facilitando el desarrollo incremental y las pruebas en vivo.

2 Arquitectura del Sistema (Estructura de Archivos)

El sistema debe estar estrictamente dividido en **4 archivos independientes**. Cada modulo tiene una responsabilidad unica y definida:

1. **admin.py**: Contiene la logica del administrador. Cada funcion en este archivo debe recibir la lista global de cursos, realizar la operacion CRUD correspondiente y retornar la lista actualizada.
 2. **usuario.py**: Contiene la logica del estudiante/usuario. Recibe la lista global de cursos disponibles y gestiona la coleccion de cursos que este decide matricular, retornando el resultado de la matricula.
 3. **auth.py**: Modulo encargado del sistema de autenticacion (Inicio de sesion y logout).
 4. **main.py**: El punto de entrada del programa. Coordina el flujo principal mediante un menu interactivo, importando las funciones de los tres modulos anteriores.
-

3 Requerimientos Generales del Sistema

3.1 Estructura de Datos de los Cursos

Para mantener el sistema simple pero estructurado, los cursos se representaran dentro de una lista global. Cada curso individual sera un **diccionario** con la siguiente estructura exacta:

- **Clave principal**: El codigo del curso (un string unico, por ejemplo: "IF-2000").
- **Valor asociado**: El nombre del profesor asignado (un string, por ejemplo: "Prof. Alberto Garbanzo").

3.2 Reglas de Negocio

- **Autenticacion fija:** Existen unicamente dos roles predefinidos con credenciales quemadas en el codigo:
 - Administrador: usuario "admin", contrasena "admin123".
 - Usuario regular: usuario "estudiante", contrasena "u123".
- **Flujo continuo:** Al iniciar el programa, se solicita el login. Una vez autenticado, el usuario ve el menu correspondiente a su rol. Cuando selecciona la opcion de logout, el sistema no termina, sino que regresa a la pantalla de inicio de sesion principal.
- **Manejo de Robustez:** El sistema debe validar que las entradas sean correctas. Por ejemplo, intentar eliminar un curso que no existe o ingresar una opcion de menu invalida no debe tumbar el programa, sino capturarse mediante excepciones o validaciones logicas.

4 Historias de Usuario y Criterios de Aceptacion

A continuacion se detallan las historias de usuario organizadas por modulos. Se incluye un espacio [] para que el estudiante pueda de forma interactiva ir tachando los requerimientos conforme complete la implementacion.

4.1 Modulo de Autenticacion (auth.py y main.py)

Historia de Usuario 1: Inicio de Sesion Diferenciado Como **usuario del sistema**, quiero **ingresar mis credenciales (usuario y contrasena)** para **acceder a las funciones especificas de mi rol (Administrador o Estudiante)**.

- [] **Criterio 1.1:** El sistema solicita de forma interactiva el usuario y la contrasena.
- [] **Criterio 1.2:** Si los datos corresponden a admin / admin123, el sistema redirige al menu del Administrador.
- [] **Criterio 1.3:** Si los datos corresponden a estudiante / u123, el sistema redirige al menu del Usuario.
- [] **Criterio 1.4:** Si las credenciales son incorrectas, muestra un mensaje de error claro y vuelve a pedir los datos sin que el programa se cierre.

Historia de Usuario 2: Cierre de Sesion (Logout) Como **usuario autenticado**, quiero **seleccionar una opcion de cerrar sesion** para **salir de mi menu actual y permitir que otro usuario inicie sesion**.

- [] **Criterio 2.1:** Tanto el menu de administrador como el de usuario tienen una opcion explicita de "Cerrar Sesion".
- [] **Criterio 2.2:** Al seleccionar la opcion, se limpia el estado actual y el sistema vuelve inmediatamente a solicitar credenciales de inicio.

4.2 Modulo del Administrador (admin.py)

Historia de Usuario 3: Crear Curso (Create) Como **administrador**, quiero **registrar un nuevo curso con su codigo y profesor** para **ampliar la oferta academica disponible**.

- [] **Criterio 3.1:** La funcion recibe la lista de cursos actual, solicita el codigo unico (ej: "MA-01") y el nombre del profesor, y la retorna modificada.
- [] **Criterio 3.2:** Se debe validar que el codigo del curso no exista previamente en la lista para evitar duplicados.

Historia de Usuario 4: Leer Cursos (Read) Como **administrador**, quiero **visualizar todos los cursos registrados** para **conocer el estado de la oferta actual**.

- [] **Criterio 4.1:** Muestra en consola de forma ordenada el listado de codigos de curso y sus respectivos profesores.
- [] **Criterio 4.2:** Si no hay cursos registrados, debe desplegar un mensaje indicando que la lista esta vacia.

Historia de Usuario 5: Actualizar Profesor de un Curso (Update) Como administrador, quiero **modificar el profesor de un curso existente** para **mantener los horarios actualizados ante cambios de personal**.

- [] **Criterio 5.1:** Se solicita el codigo del curso que se desea modificar.
- [] **Criterio 5.2:** Si el curso existe, solicita el nuevo nombre del profesor y actualiza el diccionario. Si no existe, lanza una advertencia en consola sin caer en un error fatal.

Historia de Usuario 6: Eliminar Curso (Delete) Como administrador, quiero **remove un curso del sistema mediante su codigo para dar de baja materias que no se abran**.

- [] **Criterio 6.1:** Busca el curso por su codigo dentro de la lista y lo elimina si lo encuentra.
- [] **Criterio 6.2:** Si el codigo introducido no corresponde a ningun curso registrado, se le notifica al usuario de forma clara.

4.3 Modulo del Usuario / Estudiante (usuario.py)

Historia de Usuario 7: Matricular Curso Como estudiante, quiero **ver los cursos disponibles y seleccionar uno por su codigo para agregarlo a mi lista personal de matricula**.

- [] **Criterio 7.1:** La funcion recibe la lista global de cursos y mantiene de forma interna o persistente la lista de cursos que el estudiante ha matriculado.
- [] **Criterio 7.2:** Un estudiante no puede matricular un curso que ya matriculo previamente en su sesion.
- [] **Criterio 7.3:** El estudiante puede elegir ver los cursos que ya tiene matriculados en cualquier momento de su menu.

—

5 Guia de Implementacion Tecnica (Esqueleto Base)

Para orientar el desarrollo en vivo, las firmas de las funciones en los modulos deben estructurarse de la siguiente manera:

```
# admin.py
def agregar_curso(lista_cursos):
    # Solicitar codigo y profesor...
    # Validar que no exista...
    # Cada curso es un diccionario: {codigo: profesor}
    return lista_cursos

def eliminar_curso(lista_cursos):
    # Buscar el diccionario con la clave y removerlo...
    return lista_cursos

# usuario.py
def matricular_curso(lista_cursos_disponibles, mis_cursos_matriculados):
    # Mostrar opciones, leer codigo, validar si existe en disponibles
    # y si no ha sido matriculado aun...
    return mis_cursos_matriculados
```